The Dice Game

Project Report | Java Programming, IT-Högskolan | Single-player console game

1. Introduction

The Dice Game is a single-player, text-based Java console application. The player chooses how many rounds to play and how many sides the die should have, then tries to guess the result of each roll. The program is split into three classes (**DiceGame**, **Player** and **Die**) inside the package `Game`.

2. Game Rules

- The player decides the **number of rounds** (must be greater than 0).
- The player decides the **number of sides** on the die (at least 2).
- Each round, the player guesses a whole number between 1 and the number of sides.
- The die is rolled. A **correct guess earns +1 point**; a wrong guess earns no points.
- The current score is shown after every round. After the last round the game ends and the player's name and **final score** are displayed.

3. How to Start the Game

Requires a Java JDK. From the project root folder (the folder containing `Game/`):

```
javac Game/*.java
java Game.DiceGame
```

Alternatively, open the project in an IDE (e.g. IntelliJ IDEA or VS Code) and run the `main` method in `DiceGame.java`. Then follow the prompts: enter the number of rounds, the number of die sides, your name, and one guess per round.

4. Overview of Classes and Methods

Class	Method	Purpose
DiceGame	`main(String[] args)`	Entry point. Creates the Scanner, prints the welcome message, asks for and validates the number of rounds (> 0) and die sides (≥ 2), reads the player's name, creates the Player and its Die, calls playRounds() and finally prints the GAME OVER summary with the final score.
	`getValidGuess(Scanner, int sides)`	Private helper. Repeatedly asks for a guess until it is a whole number between 1 and sides. Catches InputMismatchException for non-numeric input and clears the invalid input.
	`playRounds(Player, Scanner, int rounds, int sides)`	Private helper containing the game loop. For each round: gets a valid guess, rolls the die, shows the result, awards a point on a correct guess and prints the current score.
Player	`Player(String playerName)`	Constructor. Stores the name and starts the score at 0.
	`getPlayerName(), getScore(), setScore(int)`	Getters/setter for the player's name and score (setScore is available but not used by the game).
	`addDie(int numberOfSides)`	Creates the player's Die with the chosen number of sides.
	`rollDice(), getDieValue()`	Delegates to the Die: rolls it and returns the most recent rolled value.
	`increaseScore()`	Adds one point to the score after a correct guess.
Die	`Die(int numberOfSides)`	Constructor. Stores the number of sides, sets currentValue to 0 and creates a Random generator.
	`roll()`	Generates a random value from 1 to numberOfSides using random.nextInt(numberOfSides) + 1.
	`getCurrentValue(), getNumberOfSides()`	Return the last rolled value and the number of sides.

5. Key Coding Decisions

- **Separation of responsibilities:** Die only knows how to roll, Player owns the name, score and die, and DiceGame handles user interaction and game flow. This keeps each class small and easy to test.
- **Encapsulation:** all fields are private and accessed through methods. Player hides the Die behind rollDice() and getDieValue(), so DiceGame never touches the Die directly.
- **Input validation:** every numeric input is read inside a while-loop with try/catch on InputMismatchException, so letters or out-of-range values never crash the program; the user is simply asked again with a clear error message.
- **Scanner buffer handling:** scanner.nextLine() is called after a failed nextInt() to discard the invalid token, and once more before reading the name so the leftover newline is not read as the name.
- **Flexible die:** the number of sides is chosen by the player instead of being fixed to 6, and the guess range adapts automatically.
- **Helper methods:** getValidGuess() and playRounds() are extracted from main() (refactoring) to make main shorter and more readable. They are private static because they are only used inside DiceGame.
- **Randomness:** java.util.Random with nextInt(sides) + 1 gives an even chance for every face from 1 to sides.
- **Resource handling:** the single Scanner is shared across methods and closed at the end of main().